



UNIVERSIDAD POLITÉCNICA METROPOLITANA DE HIDALGO

Ingeniería en Tecnologías de la Información

Tecnologías y Aplicaciones en Internet

8° B

Integrantes:

Hernandez Espinosa Zaid Emanuel
Herrera Cruz Vielka Ailyn
Lopez Lara Randy Noe
Gonzalez Lopez Jorge Saul

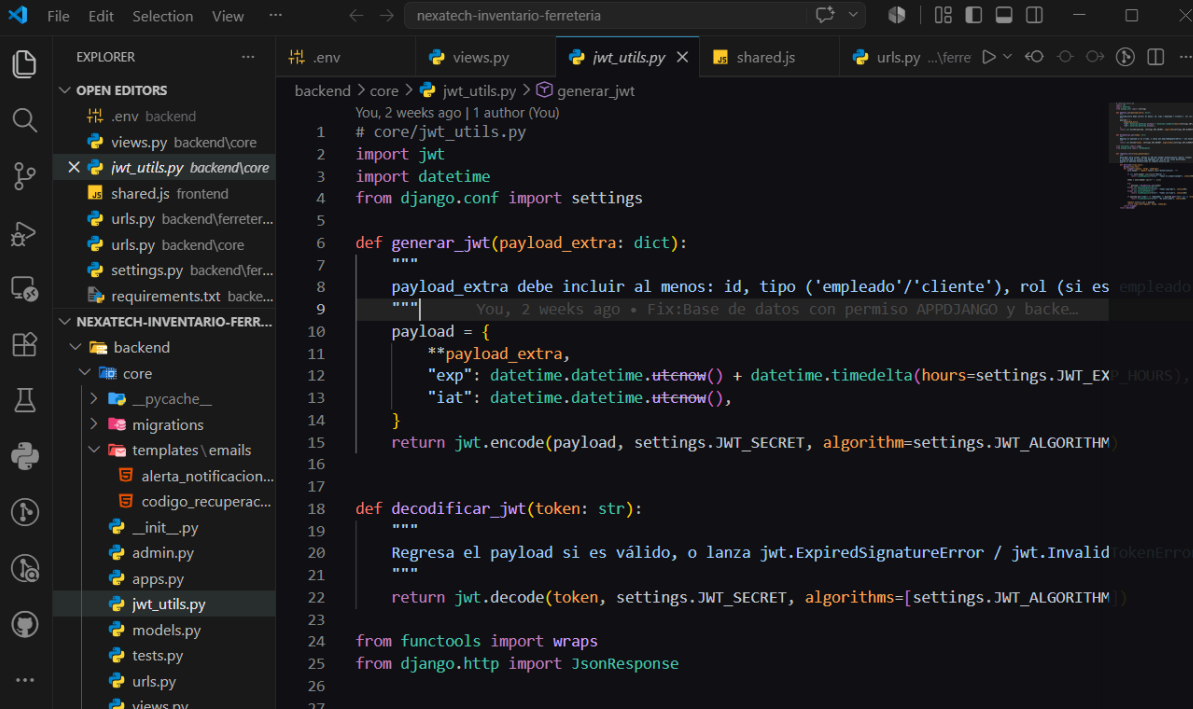
Profesor:

Dr. Carlos Alberto Carrillo Santos.

Tolcayuca, Hidalgo Ex Hacienda San Javier

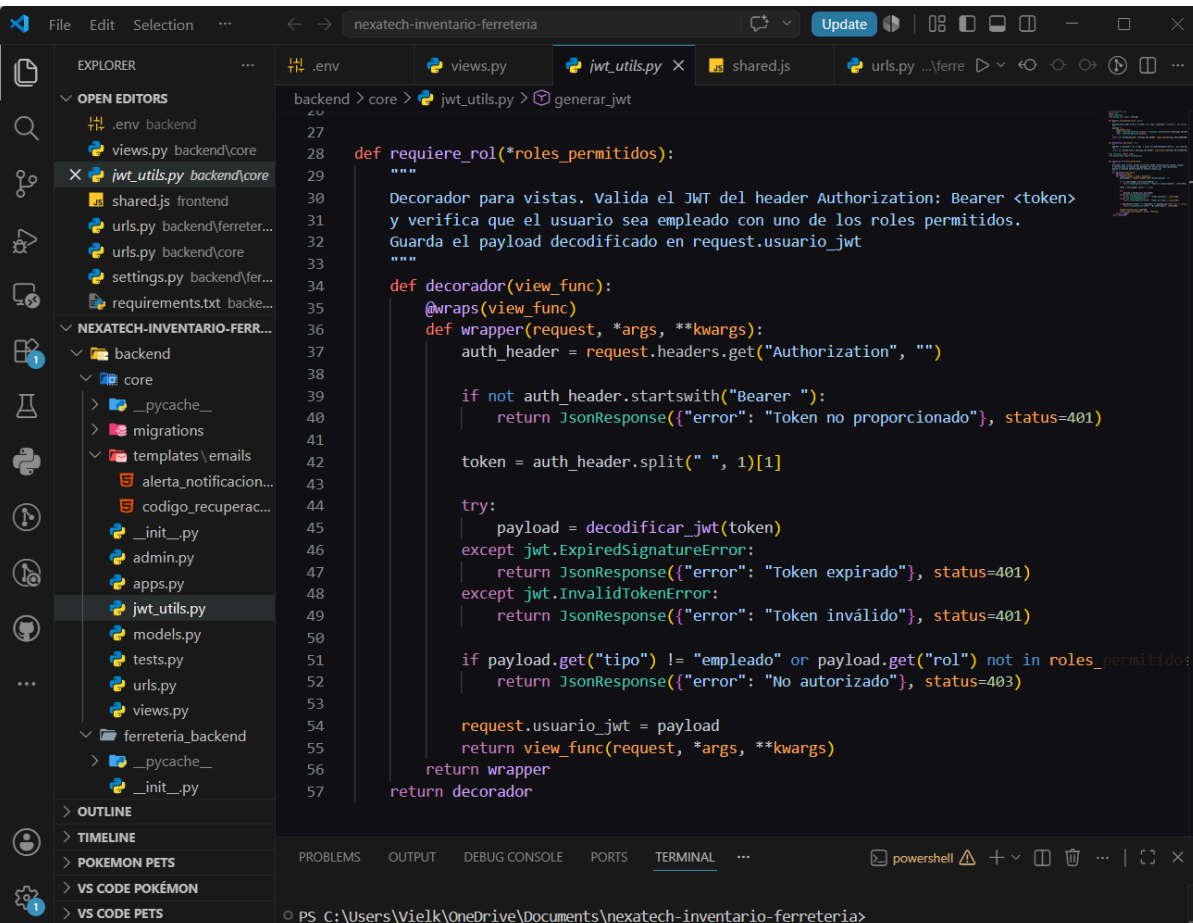
17 ago 2026

Este código se encarga de crear y validar tokens JWT para la autenticación. `generar_jwt()` crea el token con los datos del usuario y una fecha de expiración. `decodificar_jwt()` verifica que el token sea válido y no esté vencido, y luego devuelve la información del usuario.



```
File Edit Selection View ... nexatech-inventario-ferreteria
EXPLORER
  OPEN EDITORS
    .env backend
    views.py backend\core
    X jwt_utils.py backend\core
    shared.js frontend
    urls.py backend\ferreter...
    urls.py backend\core
    settings.py backend\fer...
    requirements.txt backe...
  NEXATECH-INVENTARIO-FERR...
    backend
      core
        __pycache__
        migrations
        templates\emails
          alerta_notificacion...
          codigo_recuperac...
        __init__.py
        admin.py
        apps.py
        jwt_utils.py
        models.py
        tests.py
        urls.py
        views.py

backend > core > jwt_utils.py > generar_jwt
You, 2 weeks ago | 1 author (You)
# core/jwt_utils.py
1 import jwt
2 import datetime
3 from django.conf import settings
4
5
6 def generar_jwt(payload_extra: dict):
7     """
8     payload_extra debe incluir al menos: id, tipo ('empleado'/'cliente'), rol (si es empleado)
9     """
10    You, 2 weeks ago * Fix:Base de datos con permiso APPDJANGO y backe...
11    payload = {
12        **payload_extra,
13        "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=settings.JWT_EXPIRATION_HOURS),
14        "iat": datetime.datetime.utcnow(),
15    }
16    return jwt.encode(payload, settings.JWT_SECRET, algorithm=settings.JWT_ALGORITHM)
17
18 def decodificar_jwt(token: str):
19     """
20     Regresa el payload si es válido, o lanza jwt.ExpiredSignatureError / jwt.InvalidTokenError
21     """
22    return jwt.decode(token, settings.JWT_SECRET, algorithms=[settings.JWT_ALGORITHM])
23
24 from functools import wraps
25 from django.http import JsonResponse
26
27
```



```
File Edit Selection ... nexatech-inventario-ferreteria Update
EXPLORER
  OPEN EDITORS
    .env backend
    views.py backend\core
    X jwt_utils.py backend\core
    shared.js frontend
    urls.py backend\ferreter...
    urls.py backend\core
    settings.py backend\fer...
    requirements.txt backe...
  NEXATECH-INVENTARIO-FERR...
    backend
      core
        __pycache__
        migrations
        templates\emails
          alerta_notificacion...
          codigo_recuperac...
        __init__.py
        admin.py
        apps.py
        jwt_utils.py
        models.py
        tests.py
        urls.py
        views.py
      ferreteria_backend
        __pycache__
        __init__.py

backend > core > jwt_utils.py > generar_jwt
27
28 def requiere_rol(*roles_permitidos):
29     """
30     Decorador para vistas. Valida el JWT del header Authorization: Bearer <token>
31     y verifica que el usuario sea empleado con uno de los roles permitidos.
32     Guarda el payload decodificado en request.usuario_jwt
33     """
34
35     def decorador(view_func):
36         @wraps(view_func)
37         def wrapper(request, *args, **kwargs):
38             auth_header = request.headers.get("Authorization", "")
39
40             if not auth_header.startswith("Bearer "):
41                 return JsonResponse({"error": "Token no proporcionado"}, status=401)
42
43             token = auth_header.split(" ", 1)[1]
44
45             try:
46                 payload = decodificar_jwt(token)
47             except jwt.ExpiredSignatureError:
48                 return JsonResponse({"error": "Token expirado"}, status=401)
49             except jwt.InvalidTokenError:
50                 return JsonResponse({"error": "Token inválido"}, status=401)
51
52             if payload.get("tipo") != "empleado" or payload.get("rol") not in roles_permitidos:
53                 return JsonResponse({"error": "No autorizado"}, status=403)
54
55             request.usuario_jwt = payload
56             return view_func(request, *args, **kwargs)
57         return wrapper
58     return decorador
59
```